15.1.1 Versioning release candidates for continuous delivery

The output of the commit stage of the deployment pipeline is a release candidate. That's the deployable artifact for an application. In our case, it's a container image. All the subsequent steps in the pipeline will evaluate the quality of that container image through different tests. If no issue is found, the release candidate is ultimately deployed to production and released to users.

A release candidate is stored in an artifact repository. If it's a JAR, it would be stored in a Maven repository. In our case, it's a container image and will be stored in a container registry. In particular, we'll use GitHub Container Registry.

Each release candidate must be uniquely identified. So far, we have used the implicit `latest` tag for all container image versions. Also, we ignored the `0.0.1-SNAPSHOT` version configured in each Spring Boot project by default in Gradle. How should we version release candidates?

A popular strategy is *semantic versioning* (https://semver.org). It consists of identifiers in the form of `<major>.<minor>.<patch>`. Optionally, you can also add a hyphen at the end, followed by a string, marking a pre-release. By default, a Spring Boot project generated from Spring Initializr (https://start.spring.io) is initialized with version `0.0.1-SNAPSHOT`, which identifies a snapshot release. A variation of this strategy is *calendar versioning* (https://calver.org), which combines the concepts of semantic versioning with date and time.

Both those strategies are broadly used for open source projects and software released as products to customers, because they provide implicit information